

Banco de Dados I

Aula 06 — Do Modelo Relacional ao SQL



Prof. Dr. João Choma

Universidade Estadual de Maringá — DIN/CTC

github.com/JoaoChoma

Disciplina 6892 ■ Modelo físico e SQL DDL



Até aqui, já aprendemos a:

- identificar entidades, atributos e relacionamentos;
- representar o domínio por meio do DER/MER;
- transformar o modelo conceitual em um **Modelo Relacional (MR)**;
- identificar:
 - relações/tabelas;
 - atributos;
 - chaves primárias;
 - chaves estrangeiras.

Agora surge uma nova etapa:

Como transformar esse modelo em um banco de dados que realmente existe em um SGBD?



```
MODELO CONCEITUAL  
DER / MER  
↓  
MODELO LÓGICO  
Modelo Relacional  
↓  
MODELO FÍSICO  
Implementação no SGBD  
↓  
DADOS
```

O **SQL** será a principal linguagem utilizada para realizar essa última transformação.



Considere o Modelo Relacional:

CLIENTE

idCliente (PK)

nome

email

PEDIDO

idPedido (PK)

data

valor

idCliente (FK)

No modelo lógico, sabemos **o que deve existir**.

Mas ainda precisamos decidir **como isso será implementado**.



O Modelo Relacional descreve logicamente:

```
CLIENTE(  
    idCliente,  
    nome,  
    email  
)
```

Mas um SGBD precisa de mais informações.

Por exemplo:

- idCliente é número ou texto?
- nome pode ficar vazio?
- qual o tamanho máximo do nome?
- email pode se repetir?
- como identificar a chave primária?
- como impedir referências a clientes inexistentes?

Essas decisões pertencem ao **modelo físico**.



Modelo Lógico

Modelo Físico

Define a estrutura lógica

Define como ela será implementada

Relações Tabelas

Atributos Colunas

PK e FK Restrições implementadas

Independente de detalhes do SGBD Considera recursos e tipos do SGBD

nome VARCHAR(100)

valor DECIMAL(10,2)



Uma mudança importante de perspectiva

Até agora perguntávamos:

Quais dados precisamos representar?

No modelo físico passamos também a perguntar:

Como o SGBD deve armazenar e controlar esses dados?

Portanto, não estamos abandonando a modelagem.

Estamos **implementando as decisões tomadas durante a modelagem.**



A atividade anterior preparou esta transição

Na atividade sobre integridade de dados, investigamos problemas como:

- identificadores repetidos;
- atributos obrigatórios sem valor;
- e-mails duplicados;
- preços negativos;
- estoques negativos;
- pedidos relacionados a clientes inexistentes.

A questão agora é:

Como transformar essas regras em algo que o próprio banco consiga garantir?



Regra de negócio → Restrição no banco

Considere:

Todo cliente deve possuir um identificador único.

No modelo:

idCliente (PK)

No banco:

idCliente INT PRIMARY KEY

A ideia é simples:

```
REGRA
↓
MODELO
↓
RESTRIÇÃO
↓
SQL
↓
SGBD GARANTE A REGRA
```



Regra:

Todo cliente deve possuir um nome.

Modelo lógico:

```
CLIENTE
-----
idCliente (PK)
nome
email
```

Modelo físico:

```
nome VARCHAR(100) NOT NULL
```

Agora o SGBD possui uma instrução explícita:

nome não pode receber NULL.



A atividade anterior introduziu mecanismos importantes:

Necessidade	Mecanismo
Identificador único	PRIMARY KEY
Relacionar tabelas	FOREIGN KEY
Valor obrigatório	NOT NULL
Valor não repetido	UNIQUE
Validar uma condição	CHECK
Definir valor inicial	DEFAULT

Esses mecanismos começam a transformar nosso **modelo lógico em modelo físico**.



Um atributo no modelo lógico pode ser:

```
nome
```

No modelo físico precisamos definir seu domínio computacional.

Exemplo:

```
nome VARCHAR(100)
```

Isso significa que precisamos escolher um **tipo de dado**.



Tipos de dados

Tipos comuns em bancos relacionais:

Tipo	Uso típico
INT	números inteiros
DECIMAL(p,s)	valores decimais
VARCHAR(n)	textos de tamanho variável
CHAR(n)	textos de tamanho fixo
DATE	datas
TIME	horários
TIMESTAMP	data e horário
BOOLEAN	verdadeiro/falso

Os nomes e detalhes dos tipos podem variar entre SGBDs.



Tipo de dado também protege o banco

Imagine:

```
estoque
```

Se definirmos:

```
estoque INT
```

o banco passa a esperar números inteiros.

Mas isso ainda permite:

```
10  
0  
-5
```

Portanto:

Tipo de dado e regra de domínio não são necessariamente a mesma coisa.



Tipo + restrição

Regra:

O estoque não pode ser negativo.

Podemos representar:

estoque **INT CHECK** (estoque ≥ 0)

Agora temos duas decisões:

INT↓

qual tipo de valor?

CHECK↓

quais valores são permitidos?



Modelo lógico:

PRODUTO

idProduto (PK)

nome

preco

estoque

Primeiras decisões físicas:

idProduto → INT

nome → VARCHAR(100)

preco → DECIMAL(10,2)

estoque → INT

Depois adicionamos as restrições.



```
idProduto INT PRIMARY KEY
```

```
nome VARCHAR(100) NOT NULL
```

```
preco DECIMAL(10,2) CHECK (preco >= 0)
```

```
estoque INT DEFAULT 0 CHECK (estoque >= 0)
```

Perceba:

Cada parte do SQL representa uma decisão sobre os dados.



O que é SQL?

SQL — Structured Query Language

É uma linguagem utilizada para trabalhar com bancos de dados relacionais.

Com SQL podemos:

- definir estruturas;
- inserir dados;
- consultar dados;
- alterar dados;
- remover dados;
- definir restrições;
- controlar transações;
- controlar permissões.



SQL não serve apenas para consultar

É comum associar SQL apenas a:

```
SELECT ...
```

Mas SQL também permite criar e modificar a estrutura do banco.

Por exemplo:

```
CREATE TABLE ...
```

Isso será nosso ponto inicial.



Categorias de comandos SQL

Uma organização didática comum:

SQL

+++ DDL → definição das estruturas

+++ DML → manipulação dos dados

+++ DQL → consultas

+++ DCL → controle de acesso

+++ TCL → controle de transações

*Algumas referências classificam **SELECT** dentro de **DML** em vez de usar **DQL** como categoria separada.*



Nosso fluxo será:

```
1. Modelo Relacional
   ↓
2. Modelo Físico
   ↓
3. CREATE TABLE
   ↓
4. Tipos de dados
   ↓
5. Restrições de integridade
   ↓
6. INSERT
   ↓
7. SELECT
   ↓
8. UPDATE / DELETE
```

Assim, SQL será apresentado como uma **continuação natural da modelagem**.



DDL trabalha com a **estrutura** do banco.

Comandos importantes:

```
CREATE  
ALTER  
DROP
```

Inicialmente, nosso foco será:

```
CREATE TABLE
```



Modelo lógico:

CLIENTE

idCliente (PK)

nome

email

Uma possível implementação:

```
CREATE TABLE cliente (  
    idCliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE  
);
```



Lendo SQL como uma especificação

Não tente apenas memorizar a sintaxe.

Leia o comando:

```
idCliente INT PRIMARY KEY
```

como:

Existe uma coluna chamada `idCliente`, que armazena números inteiros e identifica unicamente cada registro.



Interpretando coluna por coluna

```
nome VARCHAR(100) NOT NULL
```

Podemos ler:

A coluna nome armazena texto de até 100 caracteres e deve possuir um valor.



```
email VARCHAR(150) UNIQUE
```

Podemos ler:

A coluna `email` armazena texto e não permite valores duplicados entre os valores sujeitos à regra de unicidade do SGBD.

O tratamento de `NULL` em restrições `UNIQUE` possui particularidades entre SGBDs e será discutido quando necessário.



Modelo lógico:

PRODUTO

idProduto (PK)

nome

preco

estoque

Modelo físico:

```
CREATE TABLE produto (  
    idProduto INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10,2) CHECK (preco >= 0),  
    estoque INT DEFAULT 0 CHECK (estoque >= 0)  
);
```



Modelo:

PEDIDO

idPedido (PK)

data

valor

idCliente (FK)

Sabemos que:

PEDIDO.idCliente



CLIENTE.idCliente

Como fazer o banco garantir esse relacionamento?



A chave estrangeira transforma o relacionamento lógico em uma restrição implementada.

```
FOREIGN KEY (idCliente)  
  REFERENCES cliente(idCliente)
```

Ela estabelece que o valor informado em `PEDIDO.idCliente` deve respeitar a referência definida para `CLIENTE.idCliente`.



```
CREATE TABLE pedido (  
    idPedido INT PRIMARY KEY,  
    data DATE NOT NULL,  
    valor DECIMAL(10,2) CHECK (valor >= 0),  
    idCliente INT NOT NULL,  
  
    FOREIGN KEY (idCliente)  
        REFERENCES cliente(idCliente)  
);
```

Agora o relacionamento está implementado no banco.



Modelo Relacional:

```
CLIENTE(  
    idCliente PK,  
    nome,  
    email  
)  
  
PEDIDO(  
    idPedido PK,  
    data,  
    valor,  
    idCliente FK → CLIENTE  
)
```

Modelo físico:

```
atributos  
↓  
colunas + tipos + restrições  
↓  
CREATE TABLE
```



Considere:

```
CLIENTE ← PEDIDO
```

PEDIDO depende de CLIENTE.

Portanto, normalmente criamos:

```
1º CLIENTE  
2º PEDIDO
```

Por quê?

Porque a tabela referenciada precisa existir para que a restrição de chave estrangeira seja definida.



Depois de criar, o modelo físico pode evoluir

Criar uma tabela não significa que sua estrutura ficará imutável.

Durante a evolução de um sistema, podem surgir novas necessidades:

- adicionar um novo atributo;
- alterar uma coluna existente;
- remover um atributo;
- adicionar uma nova restrição;
- remover uma restrição;
- renomear elementos;
- remover uma tabela inteira.

Para modificar a estrutura já existente, utilizamos principalmente:

```
ALTER TABLE
```



Os principais comandos DDL que trabalharemos neste momento são:

CREATE



cria uma estrutura

ALTER



modifica uma estrutura existente

DROP



remove uma estrutura

Portanto:

CREATE TABLE → cria

ALTER TABLE → altera

DROP TABLE → remove



Criamos inicialmente:

```
CREATE TABLE cliente (  
    idCliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(150) UNIQUE  
);
```

Depois percebemos uma nova necessidade:

Precisamos armazenar o telefone do cliente.

Não precisamos remover e recriar toda a tabela.

Podemos alterar sua estrutura.



Podemos adicionar uma nova coluna com:

```
ALTER TABLE cliente  
ADD telefone VARCHAR(20);
```

Agora a estrutura passa de:

CLIENTE

idCliente
nome
email

para:

CLIENTE

idCliente
nome
email
telefone



Antes:

```
CLIENTE(  
    idCliente,  
    nome,  
    email  
)
```

Depois:

```
CLIENTE(  
    idCliente,  
    nome,  
    email,  
    telefone  
)
```

No SQL:

```
ALTER TABLE cliente  
ADD telefone VARCHAR(20);
```

O ALTER TABLE modifica uma estrutura que já existe no banco.



Imagine que o telefone passe a ser obrigatório.

A necessidade deixa de ser apenas:

armazenar telefone.

e passa a ser:

todo cliente deve possuir telefone.

Isso implica modificar a definição da coluna.

A sintaxe exata para alteração de tipo, nulabilidade ou definição de coluna pode variar entre SGBDs.



ALTER TABLE — alterando uma coluna

Em SGBDs que utilizam a sintaxe ALTER COLUMN, uma alteração pode assumir a forma:

```
ALTER TABLE cliente  
ALTER COLUMN telefone VARCHAR(20) NOT NULL;
```

Em outros SGBDs, a sintaxe pode ser diferente.

Por exemplo, alguns utilizam:

```
MODIFY
```

Por isso, ao trabalhar com DDL, é importante conhecer o SGBD adotado na disciplina.



Suponha que já existam clientes sem telefone.

Depois tentamos tornar a coluna obrigatória:

```
telefone → NOT NULL
```

Surge um problema:

```
DADOS EXISTENTES
  ↓
alguns possuem telefone NULL
  ↓
NOVA REGRA
telefone NOT NULL
  ↓
CONFLITO
```

Alterar a estrutura pode exigir verificar os dados existentes.



O modelo físico e os dados estão relacionados

Uma mudança estrutural pode afetar registros já armazenados.

Exemplo:

ANTES
telefone pode ser NULL

DEPOIS
telefone NOT NULL

Antes de aplicar a alteração, podemos precisar:

- 1 localizar registros sem telefone;
- 2 corrigir esses registros;
- 3 somente então aplicar a nova restrição.

Portanto:

Evoluir o modelo físico exige considerar a estrutura e os dados existentes.



Imagine que decidimos não armazenar mais telefone.

Podemos remover a coluna:

```
ALTER TABLE cliente  
DROP COLUMN telefone;
```

Depois da operação:

```
CLIENTE  
-----  
idCliente  
nome  
email
```



Quando executamos:

```
ALTER TABLE cliente  
DROP COLUMN telefone;
```

não estamos apagando apenas os valores dos telefones.

Estamos removendo:

```
a própria coluna  
+  
todos os dados armazenados nela
```

Por isso, alterações DDL precisam ser realizadas com cuidado.



Considere a tabela:

```
CREATE TABLE produto (  
    idProduto INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    preco DECIMAL(10,2),  
    estoque INT  
);
```

Depois percebemos a regra:

O preço não pode ser negativo.

Podemos acrescentar uma restrição posteriormente.



Adicionando uma restrição

Podemos criar uma restrição nomeada:

```
ALTER TABLE produto  
ADD CONSTRAINT ck_produto_preco  
CHECK (preco >= 0);
```

Agora o SGBD passa a verificar:

```
preco >= 0
```

sempre que a regra for aplicável às operações realizadas sobre a tabela.



Por que nomear restrições?

Observe:

```
CONSTRAINT ck_produto_preco  
CHECK (preco >= 0)
```

O nome:

```
ck_produto_preco
```

permite identificar a restrição.

Uma convenção possível:

```
pk_ → PRIMARY KEY  
fk_ → FOREIGN KEY  
uk_ → UNIQUE  
ck_ → CHECK
```

Exemplos:

```
pk_cliente  
fk_pedido_cliente  
uk_cliente_email  
ck_produto_preco
```



Uma tabela poderia ser criada assim:

```
CREATE TABLE cliente (  
    idCliente INT,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(150),  
  
    CONSTRAINT pk_cliente  
        PRIMARY KEY (idCliente),  
  
    CONSTRAINT uk_cliente_email  
        UNIQUE (email)  
);
```

A estrutura é conceitualmente equivalente ao uso das restrições diretamente nas colunas, mas os nomes facilitam a identificação e manutenção.



Também podemos criar:

```
CREATE TABLE pedido (  
    idPedido INT,  
    data DATE NOT NULL,  
    valor DECIMAL(10,2),  
    idCliente INT NOT NULL,  
  
    CONSTRAINT pk_pedido  
        PRIMARY KEY (idPedido),  
  
    CONSTRAINT fk_pedido_cliente  
        FOREIGN KEY (idCliente)  
        REFERENCES cliente(idCliente)  
);
```

Agora a relação possui uma restrição explicitamente identificada.



Suponha que PEDIDO tenha sido criado inicialmente sem a FK:

```
CREATE TABLE pedido (  
    idPedido INT PRIMARY KEY,  
    data DATE NOT NULL,  
    valor DECIMAL(10,2),  
    idCliente INT NOT NULL  
);
```

Depois queremos implementar o relacionamento com CLIENTE.

Podemos utilizar:

```
ALTER TABLE pedido  
ADD CONSTRAINT fk_pedido_cliente  
FOREIGN KEY (idCliente)  
REFERENCES cliente(idCliente);
```



Imagine que PEDIDO já possua:

idPedido	idCliente
100	1
101	2
102	999

E que não exista:

```
CLIENTE.idCliente = 999
```

Ao tentar adicionar a chave estrangeira, o banco pode rejeitar a alteração.

Por quê?

Porque os dados existentes já violam a regra que queremos introduzir.



NOVA FOREIGN KEY



SGBD verifica os dados



encontra idCliente = 999



cliente 999 não existe



restrição não pode ser satisfeita

Isso mostra novamente a ligação entre:

MODELO

+

RESTRIÇÕES

+

DADOS



Removendo uma restrição

Se uma restrição deixou de fazer sentido, podemos removê-la.

Exemplo:

```
ALTER TABLE pedido  
DROP CONSTRAINT fk_pedido_cliente;
```

Depois disso, o banco deixa de garantir aquela relação por meio dessa restrição.

A sintaxe para remoção de determinadas restrições pode variar entre SGBDs.



Antes:

```
PEDIDO.idCliente  
    ↓  
FOREIGN KEY  
    ↓  
CLIENTE.idCliente
```

Depois de remover a FK:

```
PEDIDO.idCliente  
    ↓  
sem a restrição
```

A coluna continua existindo.

O que foi removido foi a **regra que garantia a referência**.



Compare:

```
ALTER TABLE pedido  
DROP CONSTRAINT fk_pedido_cliente;
```

Remove:

a regra de relacionamento

Enquanto:

```
ALTER TABLE pedido  
DROP COLUMN idCliente;
```

remove:

a própria coluna

São operações estruturalmente diferentes.



Enquanto ALTER TABLE modifica uma tabela existente:

```
DROP TABLE
```

remove a tabela.

Exemplo:

```
DROP TABLE produto;
```

Essa operação remove a estrutura da tabela e, em condições normais de uso, seus dados armazenados.

DROP TABLE



```
DROP TABLE produto;
```

Podemos interpretar:

Remova do banco a tabela produto.

Depois disso:

```
PRODUTO
```

deixa de fazer parte do esquema do banco.



Considere:

```
PRODUTO
```

```
-----  
10  Teclado  
11  Mouse  
12  Monitor
```

Ao remover a tabela:

```
DROP TABLE produto;
```

não estamos removendo apenas os três registros.

Estamos removendo:

```
estrutura  
+  
colunas  
+  
restrições  
+  
dados
```



Na criação:

```
CLIENTE ← PEDIDO
```

criamos normalmente:

```
1º CLIENTE  
2º PEDIDO
```

Na remoção, a dependência precisa ser considerada novamente.

Se PEDIDO referencia CLIENTE, tentar remover CLIENTE primeiro pode ser impedido pelo SGBD.



Uma forma de pensar:

criação

cliente



pedido

Mas para remover:

remoção

pedido



cliente

Primeiro removemos a estrutura dependente.

Depois, a estrutura referenciada.



Se temos:

```
CLIENTE ← PEDIDO
```

uma sequência possível é:

```
DROP TABLE pedido;
```

```
DROP TABLE cliente;
```

Assim, eliminamos primeiro a tabela que possui a chave estrangeira.



Outra possibilidade: remover primeiro o relacionamento

Também poderíamos remover a restrição:

```
ALTER TABLE pedido  
DROP CONSTRAINT fk_pedido_cliente;
```

e somente depois remover a tabela referenciada.

Exemplo:

```
DROP TABLE cliente;
```

Mas isso altera a estrutura de PEDIDO, que permaneceria sem a garantia do relacionamento.



Podemos pensar no DDL como operações sobre a vida estrutural do banco:

CREATE



estrutura nasce

ALTER



estrutura evolui

DROP



estrutura deixa de existir

Esses comandos trabalham com o **esquema** do banco.



O que significa esquema?

Neste contexto, o esquema representa a estrutura definida para o banco:

- tabelas;
- colunas;
- tipos;
- chaves;
- relacionamentos;
- restrições.

DDL trabalha principalmente com essa estrutura.

DDL
↓
ESQUEMA



Compare:

```
ALTER TABLE cliente  
ADD telefone VARCHAR(20);
```

Isso altera:

ESTRUTURA

Já uma operação que modifica o telefone de um cliente específico altera:

DADOS

Essa diferença será importante quando avançarmos para DML.



DDL

CREATE

ALTER

DROP

atua principalmente sobre↓

ESTRUTURA

DML

INSERT

UPDATE

DELETE

atua principalmente sobre↓

DADOS

Essa separação ajuda a compreender os próximos comandos.



Atenção ao termo UPDATE

No SQL existe o comando:

UPDATE

Mas ele **não altera a estrutura da tabela**.

Ele altera os valores armazenados em registros.

Para alterar a estrutura utilizamos:

ALTER TABLE

Portanto:

ALTER TABLE → modifica estrutura

UPDATE → modifica dados



Da mesma forma:

`DELETE`

remove registros de uma tabela.

Enquanto:

`DROP TABLE`

remove a própria tabela.

Compare:

`DELETE↓`

remove linhas

`DROP TABLE↓`

remove estrutura + dados



Objetivo	Comando
Criar uma tabela	CREATE TABLE
Adicionar uma coluna	ALTER TABLE ... ADD
Remover uma coluna	ALTER TABLE ... DROP COLUMN
Adicionar uma restrição	ALTER TABLE ... ADD CONSTRAINT
Remover uma restrição	ALTER TABLE ... DROP CONSTRAINT
Remover uma tabela	DROP TABLE
Alterar dados de registros	UPDATE
Remover registros	DELETE



Temos:

CLIENTE

idCliente

nome

email

Agora surgiram três novas necessidades:

- 1 armazenar a data de nascimento;
- 2 impedir que o e-mail fique vazio;
- 3 remover o atributo nome.

Qual operação estrutural seria necessária em cada caso?

Adicionar?

Alterar?

Remover?



Considere:

CLIENTE

idCliente PK

PEDIDO

idPedido PK

idCliente

A tabela PEDIDO foi criada sem a chave estrangeira.

Escreva o comando necessário para adicionar posteriormente o relacionamento entre as tabelas.



Considere:

CLIENTE ← PEDIDO

Você precisa remover as duas tabelas.

Qual sequência é mais adequada?

A) CLIENTE → PEDIDO

B) PEDIDO → CLIENTE

Explique usando o conceito de dependência entre chave estrangeira e tabela referenciada.



Até agora, podemos representar o fluxo:

```
MODELO RELACIONAL
  ↓
MODELO FÍSICO
  ↓
CREATE TABLE
  ↓
tabelas + tipos + restrições
  ↓
ALTER TABLE
  ↓
evolução estrutural
  ↓
DROP TABLE
  ↓
remoção estrutural
```




O que já conseguimos fazer?

Com DDL, já conseguimos:

- criar tabelas;
- definir tipos de dados;
- criar chaves primárias;
- criar chaves estrangeiras;
- estabelecer restrições;
- adicionar colunas;
- remover colunas;
- adicionar restrições;
- remover restrições;
- remover tabelas.

Agora temos condições de construir e evoluir o **esquema físico** do banco.



Próxima etapa: os dados

Até este ponto trabalhamos principalmente com:

ESTRUTURA

Agora passaremos a trabalhar com:

DADOS

A próxima pergunta será:

Como inserir registros nas tabelas que acabamos de criar?

O primeiro comando será:

INSERT



Quando começarmos a manipular registros, teremos:

```
CREATE → INSERT  
READ   → SELECT  
UPDATE → UPDATE  
DELETE → DELETE
```

Esse é o CRUD aplicado aos dados.

Mas é importante não confundir:

CRUD dos dados

com:

DDL da estrutura



Estrutura do banco

```
CREATE TABLE  
ALTER TABLE  
DROP TABLE
```

Dados armazenados

```
INSERT  
SELECT  
UPDATE  
DELETE
```

Primeiro construímos o recipiente.

Depois começamos a trabalhar com o conteúdo.



Fechamento

Agora estamos prontos para avançar para a manipulação dos registros.